



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

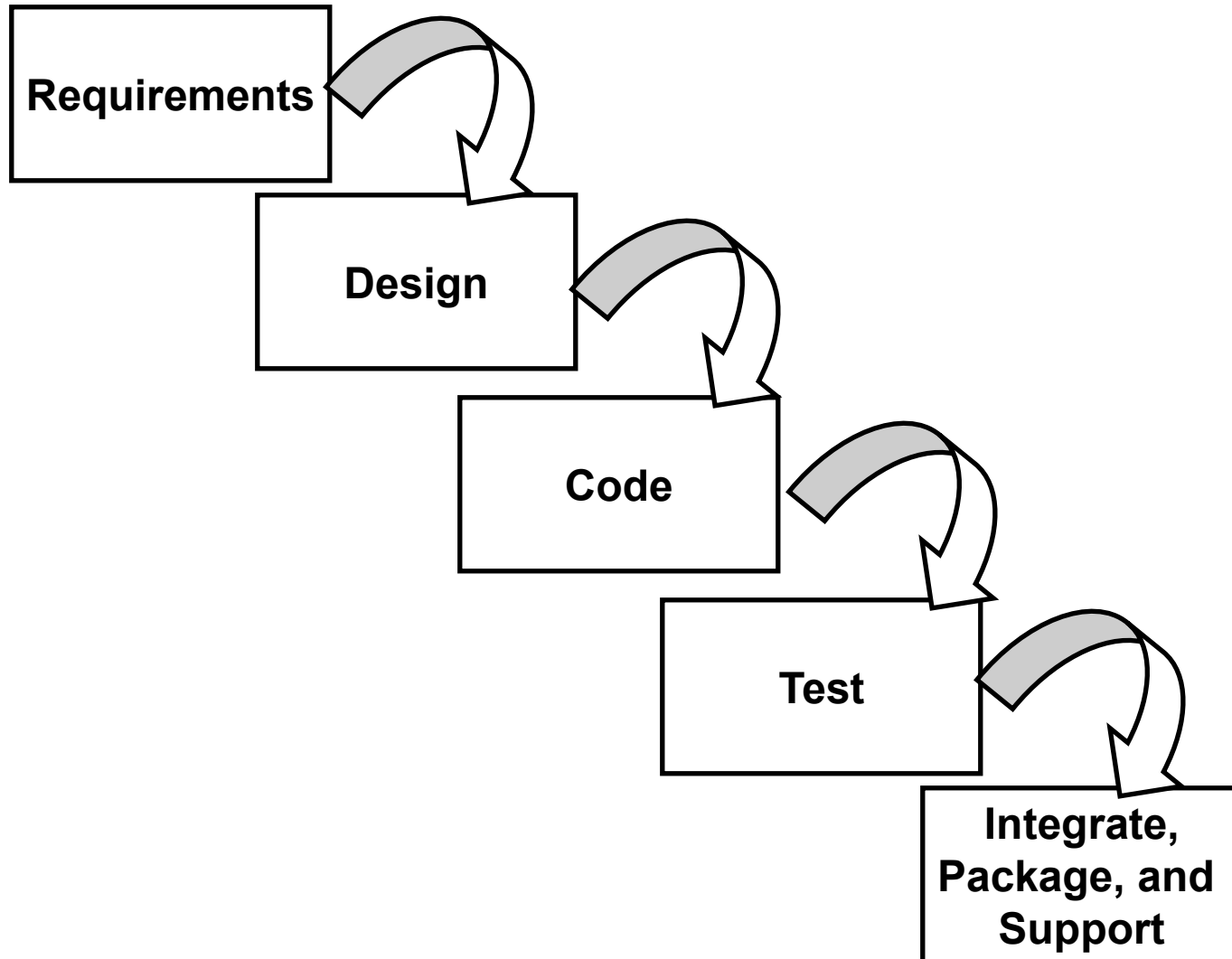
Implementation

Francesco Guerra
francesco.guerra@unimore.it

Objectives

- ▶ Describe some of the characteristics of good implementations and the best practices that help achieve them.
- ▶ Understand the role of comments in producing a good program and the characteristics of good comments.
- ▶ Learn some of the techniques for effectively debugging programs.
- ▶ Analyze the concept of refactoring and a few refactoring techniques

The waterfall model



Introduction to Implementation

- ▶ The goal of software engineering projects is to produce a working program.
- ▶ The implementation is the act of transforming the detailed design into a program in some programming language.
 - ▶ We assume an explicit, detailed design being produced.
 - ▶ In many cases the detailed design is left as part of the implementation.
 - ▶ It may result in a less cohesive and organized design: the detailed design of each module can be done by a different person.
 - In small projects, the detailed design is usually left as part of the implementation.
 - In larger projects, or when the programmers are inexperienced, the detailed design is done by a separate person.

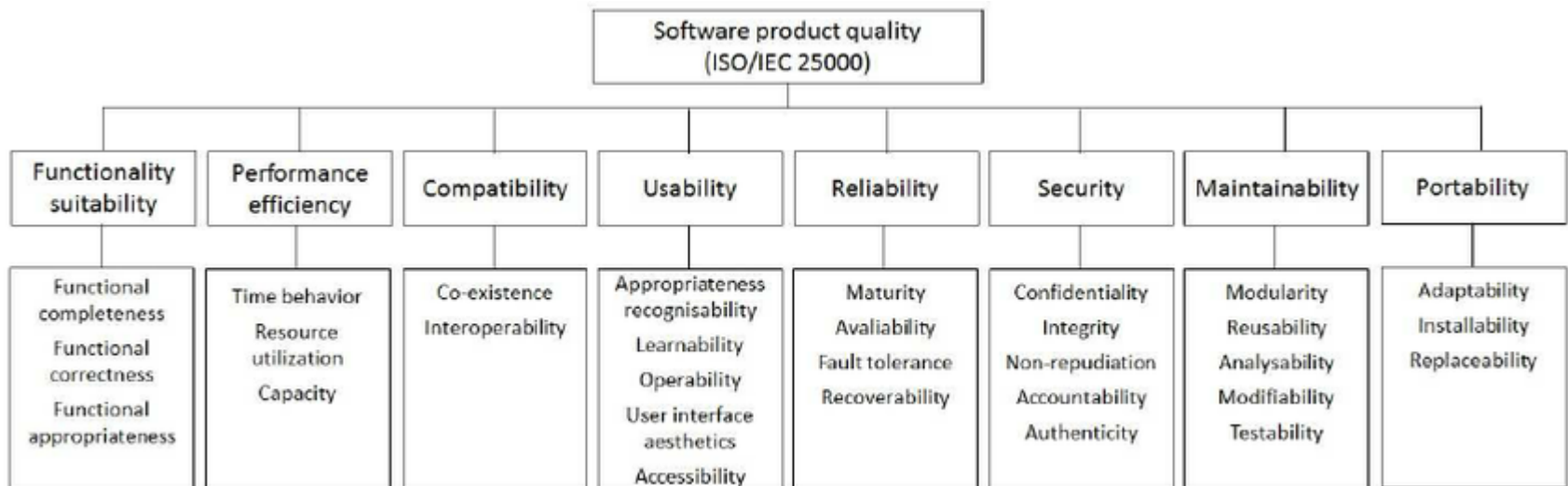
ISO/IEC 25010:2011

- ▶ The standard defines two models : Now ISO/IEC 25002:2024!
- ▶ **Software product quality model:** This model is composed of eight characteristics divided into subcharacteristics. The quality attributes derived from the model can be measured internally or externally.
 - ▶ Internally : measures static attributes by inspecting and analysing the software (need docs, sources). It provides a white box overview of the quality. It can be used during the development.
 - ▶ Externally : measures dynamic attributes by executing the software (need only binary). It provide a blackbox overview of the quality.
- ▶ **Quality in use model:** This model is composed of five characteristics divided into subcharacteristics. The attributes are measured when the software is tested in a realistic context of use. By introducing the notion of context of use, the attributes depend on the software but also on the user's, platform's and environment's characteristics.

ISO/IEC 25010:2011

Software product quality model

- ▶ The international standard ISO/IEC 25010:2011 breaks down software quality into eight characteristics:



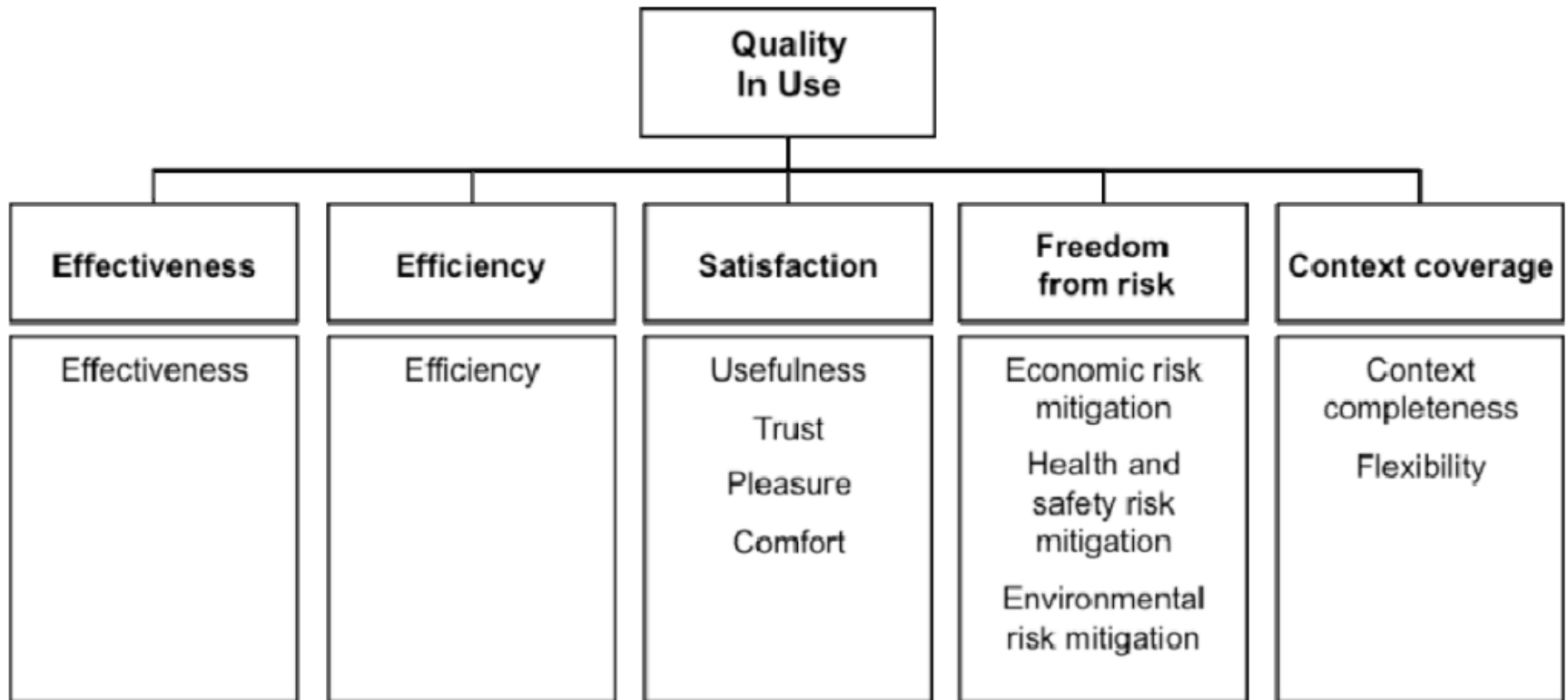
ISO/IEC 25010:2011

Software product quality model

- ▶ **Functional suitability** : “The degree to which the product provides functions that meet stated and implied needs when the product is used under specified conditions.”.
- ▶ **Performance efficiency** : “The performance relative to the amount of resources used under stated conditions.”.
- ▶ **Compatibility** : “The degree to which two or more systems or components can exchange information and/or perform their required functions while sharing the same hardware or software environment.”.
- ▶ **Usability** : “The degree to which a product or system can be used by specified users to achieve specified goals with effectiveness, efficiency and satisfaction in a specified context of use.”.
- ▶ **Reliability** : “The degree to which a system or component performs specified functions under specified conditions for a specified period of time.”.
- ▶ **Security** : “The degree to which a product or system protects information and data so that persons or other products or systems have the degree of data access appropriate to their types and levels of authorization.”.
- ▶ **Maintainability** : “The degree of effectiveness and efficiency with which the product can be modified by the intended maintainers.”.
- ▶ **Portability** : “The degree to which a system or component can be effectively and efficiently transferred from one hardware, software or other operational or usage environment to another.”.

ISO/IEC 25010:2011

Software quality in use model



ISO/IEC 25010:2011

Software quality in use model

- ▶ **Effectiveness** : “The accuracy and completeness with which users achieve specified goals”
- ▶ **Efficiency** : “The resources expended in relation to the accuracy and completeness with which users achieve goals”.
- ▶ **Satisfaction** : “The degree to which user needs are satisfied when a product is used in a specified context of use”
 - ▶ Usefulness: “The degree to which a user is satisfied with their perceived achievement of pragmatic goals, including the results of use and the consequences of use”.
 - ▶ Pleasure : “The degree to which a user obtains pleasure from fulfilling their personal needs”.
 - ▶ Comfort : “The degree to which the user is satisfied with physical comfort”.
 - ▶ Trust : “The degree to which a user or other stakeholder has confidence that a product or system will behave as intended”.
- ▶ **Freedom from risk** : “The degree to which a product or system mitigates the potential risk to economic status, human life, health, or the environment”
 - ▶ Economic damage risk : “The degree to which a product or system mitigates the potential risk to financial status, efficient operation, commercial property, reputation or other resources in the intended contexts of use”.
 - ▶ Health and safety risk : “The degree to which a product or system mitigates the potential risk to people in the intended contexts of use”.
 - ▶ Environmental risk : “The degree to which a product or system mitigates the potential risk to property or the environment in the intended contexts of use”.
- ▶ **Context coverage** : “The degree to which a product or system can be used with **effectiveness, efficiency, freedom from risk and satisfaction** in both specified contexts of use and in contexts beyond those initially explicitly identified”.
 - ▶ Context completeness : “The degree to which a product or system can be used with effectiveness, efficiency, freedom from risk and satisfaction in all the specified contexts of use”.
 - ▶ Flexibility : “The degree to which a product or system can be used with effectiveness, efficiency, freedom from risk and satisfaction in contexts beyond those initially specified in the requirements”.

Characteristics of a Good Implementation

- ▶ The implementation phase involves more than just writing code.
 - ▶ Code needs to be tested, debugged, compiled and built into an executable product.
 - ▶ Using configuration management to keep track of versions of the code
- ▶ Features of a good implementation
 - ▶ **Readability:** The code can be easily read and understood by other programmers.
 - ▶ **Maintainability:** The code can be easily modified and maintained.
 - ▶ **Performance:** All other things being equal, the implementation should produce code that performs as fast as possible.
 - ▶ **Traceability:** All code elements should correspond to a design element. Code can be traced back to design (and design to requirements).
 - ▶ **Correctness:** The implementation should do what it is intended to do (as defined in the requirements and detailed design).
 - ▶ **Completeness:** All of the system requirements are met.

Characteristics of a Good Implementation

- ▶ The first instinct of programmers is to concentrate on
 - ▶ correctness,
 - ▶ performance
 - ▶ No emphasis on readability and maintainability.
- ▶ For many software engineers engaged in large, multiple-release software projects, maintainability is as important as correctness
- ▶ Readability helps maintainability
- ▶ Readability and maintainability help achieve correctness.
- ▶ Performance optimizations can reduce readability, maintainability

Programming Style and Coding Guidelines

- ▶ Almost all software development organizations have some coding guidelines specifying
 - ▶ Naming, indentation, and commenting styles
 - ▶ Tools that automatically indent and reformat the code
 - ▶ Consistency in notation
- ▶ In (large) software projects, there are usually some programming conventions.
- ▶ The recommendation of banning certain language features and practices that have proved, for the organization, error-prone.
 - ▶ For example,
 - ▶ use of multiple inheritance in languages that support them,
 - ▶ the code has to compile without warnings and deprecated language features.

Programming Style and Coding Guidelines

- ▶ Challenge: to be consistent and to highlight the meaning of the code.
- ▶ Naming: choosing names for classes, methods, variables, and other entities.
 - ▶ Naming is mainly a semantic issue about choosing good names according to the same rules (consistency).
 - ▶ It improves readability and maintainability.
 - ▶ This is a more complex issue in multi-cultural and multi-language teams.
 - ▶ Separating words and capitalization
 - ▶ Indentation and spacing (both spaces and blank lines inserted in the code)

Programming Style and Coding Guidelines (2)

- ▶ **Function/Method Size**
- ▶ **Observation:** Large functions or methods are statistically **more error-prone** than smaller ones.
- ▶ **Rationale:**
 - ▶ Large methods are harder to **understand, test, and maintain**.
 - ▶ Developers are more likely to **introduce or overlook bugs** in long functions due to cognitive overload.
- ▶ **Practical consideration:**
 - ▶ Readability decreases once the function exceeds what can be seen **on one screen or one printed page**.
 - ▶ Being able to grasp the **entire logic at once** supports debugging and modification.
- ▶ **Good practice:**
 - ▶ Split large functions into **smaller, cohesive** ones with **single responsibility**.
 - ▶ Use **code metrics** (e.g., function length, cyclomatic complexity) to monitor this.

Comments

- ▶ Comments can significantly aid or hurt readability and maintainability.
- ▶ They may
 - ▶ distract from the actual code
 - ▶ make the program more difficult to read
 - ▶ be wrong
- ▶ Comments may become outdated as the code changes

Comments (2)

► Six different types of comments:

1. Repeat of the code: These kinds of comments should be avoided.

```
// increment i by one
++i;
```

 - Comments that add no information have negative value because they:
 - add visual clutter
 - take time to write and read
 - can become out-of-date
2. Explanation of the code: programmers explain code in human language.
 - If the code is so complex, then it should typically be rewritten.
 - In some cases, it is enough to give proper names to variables.

```
private static Node getBestChildNode(Node node) {
    Node n; // best child node candidate
    for (Node node: node.getChildren()) {
        // update n if the current state is better
        if (n == null || utility(node) > utility(n)) {
            n = node;
        }
    }
    return n;
}
```

<https://stackoverflow.blog/2021/12/23/best-practices-for-writing-code-comments/>

```
private static Node getBestChildNode(Node node) {
    Node bestNode;
    for (Node currentNode: node.getChildren()) {
        if (bestNode == null || utility(currentNode) > utility(bestNode)) {
            bestNode = currentNode;
        }
    }
    return bestNode;
}
```

Comments (3)

3. Marker in the code:

- ▶ To indicate incomplete items, improvements, and similar information.
- ▶ Using a consistent notation for these markers
- ▶ Sometimes markers to keep track of changes: better a tracked with version management software

4. Summary of the code: Comments that summarize what the code does, rather than just repeating it

- ▶ They are helpful in understanding the code, but they need to be kept up to date.

Comments (4)

5. Description of the code intent: They describe what the code is supposed to do rather than what it does.
 6. External references: These are comments that link the code to external entities, usually books or other programs.
 - ▶ Many times these can be viewed as a kind of intent statement, as in, “This function implements the XYZ algorithm, as explained in . . .,”
- ▶ The trade-off that comments imply should be recognized.
 - ▶ Comments can help clarify code and relate it to other sources
 - ▶ But they also represent some level of duplication of the code.
 - ▶ Effort is invested in their creation and, above all, in their maintenance.
 - ▶ A comment that does not correspond to the actual code that it accompanies can cause errors that are very hard to find and correct.

Comments (5)

- ▶ Comments can (wrongly) be used to justify bad coding practices.
 - ▶ Programmers produce code that is too complicated to maintain, and add comments to it, rather than rewrite it to good standards.
 - ▶ Many experts recommend producing what is called "self-documented code"
 - ▶ If the code cannot be abstracted and it is still complex, summary comments may be appropriate.
 - ▶ Code explanations and markers should only be used as temporary measures, and repetitions of the code should be always avoided.
- ▶ Wrong examples:
 - ▶ Programming books and tutorials, because they are geared for beginners, tend to provide too many comments, repeating or explaining the code.

Debugging

- ▶ It is the act of locating and fixing errors in code.
 - ▶ Errors are usually discovered through testing
 - ▶ They can be found by code inspections and normal use of the program
- ▶ Debugging is a highly iterative process:
 - ▶ creating a hypothesis about what causes the errors,
 - ▶ writing test cases to prove or disprove the hypothesis,
 - ▶ changing the code to try to fix the problem.



The Four Phases of the Debugging Process

1. Stabilization (or Reproduction)

- ▶ The goal is to **reproduce the error** in a controlled environment.
- ▶ The developer tries to **understand the conditions** that led to the failure.
- ▶ A **minimal and repeatable test case** is created to trigger the error.
- ▶ The output of this phase is a **set of test cases** that consistently reproduce the problem.

2. Localization

- ▶ This phase aims to **identify the part of the code** responsible for the error.
- ▶ Developers analyze **logs, stack traces, variable states**, or use debugging tools.
- ▶ If the stabilization phase produced a simple test case, the cause may become obvious.
- ▶ This is often the **most difficult and time-consuming** step of debugging.

The Four Phases of the Debugging Process

3. Correction

- ▶ The code is **modified to fix** the identified fault.
- ▶ It is essential to **fully understand the cause** before making changes.
- ▶ A common mistake is to apply random changes without clear diagnosis
 - ▶ this can **introduce new defects** instead of fixing the original one.
- ▶ The result is a **corrected version** of the code, ready for validation.

4. Verification

- ▶ Ensures that the **error has been removed** and that **no new errors** have been introduced.
- ▶ Developers **re-run the original test cases** and perform **regression testing**.
- ▶ The goal is to confirm that the system behaves correctly after correction.
- ▶ The outcome is a **stable, verified system**.

Errors and Debugging Tools

- ▶ **Syntax errors:** when the program violates the language grammar rules.
 - ▶ Detected by the compiler or interpreter before execution.
- ▶ **Logic errors:** the program runs but produces **incorrect or unexpected results**.
 - ▶ Harder to detect; often discovered during testing or use.
- ▶ **Finding Errors**
 - ▶ Follow **rules of thumb** to locate likely error sources.
 - ▶ Develop your own **heuristics** about which language features or program parts are **error-prone**.
 - ▶ For example: complex conditionals, boundary cases, type conversions, or resource management.
- ▶ **Tools Supporting Debugging**
 - ▶ **Source code comparators:** Highlight changes between code versions.
 - ▶ **Extended checkers:** Detect syntax, logic, or style issues (e.g., linters, static analyzers).
 - ▶ **Interactive debuggers:** Let you inspect variables, step through code, set breakpoints, and pause execution.
 - ▶ **Instrumented or safeguarded libraries:** Replace standard libraries with safer versions to detect or prevent errors.
 - ▶ **Profilers and coverage tools:** Provide insights into performance, execution paths, and test completeness—often revealing hidden issues.

Assertions and Defensive Programming

- ▶ A very useful technique is the use of assertions
 - ▶ It is related to the concepts of preconditions and postconditions.
 - ▶ A precondition is a condition that your module requires in order to produce correct results.
 - ▶ A postcondition is a condition should hold true after executing your code, if the preconditions were met.
 - ▶ Preconditions and postconditions can prove that a code executes correctly.
- ▶ It is a good practice to make your preconditions explicit by the use of assertions
 - ▶ statements that check a condition and produce an error if the condition is not met.
- ▶ Most modern programming languages have specific facilities for assertions
(<http://docs.oracle.com/javase/8/docs/technotes/guides/language/assert.html>)

Assertion

- ▶ The assertion statement has two forms.
 - ▶ `assert Expression1 ;`
where *Expression*₁ is a boolean expression.
When the system runs the assertion, it evaluates *Expression*₁ and if it is false throws an [AssertionError](#) with no detail message.
 - ▶ `assert Expression1 : Expression2 ;`
where: *Expression*₁ is a boolean expression.
*Expression*₂ is an expression that has a value.
 - ▶ Use this version of the assert statement to provide a detail message for the `AssertionError`.
 - ▶ The system passes the value of *Expression*₂ to the appropriate `AssertionError` constructor, which uses the string representation of the value as the error's detail message.

Putting Assertions Into Your Code

- ▶ There are many situations where it is good to use assertions, including:
 - ▶ Internal Invariants
 - ▶ Control-Flow Invariants
 - ▶ Preconditions, Postconditions, and Class Invariants

Internal Invariants

```
if (i % 3 == 0) {  
    ...  
} else if (i % 3 == 1) {  
    ...  
} else {  
    assert i % 3 == 2 : i;  
    ...  
}
```

```
switch(suit) {  
    case Suit.CLUBS:  
        ...  
        break;  
  
    case Suit.DIAMONDS:  
        ...  
        break;  
  
    case Suit.HEARTS:  
        ...  
        break;  
  
    case Suit.SPADES:  
        ...  
        break;  
    default:  
        assert false;  
}
```

- ▶ The first assertion may fail if i is negative, as the `%` operator is not true *modulus* operator, but computes the *remainder*, which may be negative.
- ▶ The second assertion indicates an assumption that the `suit` variable will have one of only four values.

Control-Flow Invariants

- ▶ The previous example also checks an assumption on the application's flow of control.
- ▶ The author of the original switch statement assumed not only that the suit variable would always have one of four values, but also that one of the four cases would always be executed.
- ▶ It points out another general area where you should use assertions: **place an assertion at any location you assume will not be reached.**
- ▶ The assertions statement to use is:
 - ▶ `assert false;`

Control-Flow Invariants

Example

```
void foo() {  
    for (...) {  
        if (...)  
            return;  
    }  
    // Execution should never reach this point!!!  
}  
  
void foo() {  
    for (...) {  
        if (...)  
            return;  
    }  
    assert false; // Execution should never reach this point!  
}
```

- ▶ Use this technique with discretion. If a statement is unreachable as defined in the Java Language Specification, you will get a compile time error if you try to assert that it is not reached.

```
class Test {  
    public static void main(String args[]) {  
        System.out.println("hello world");  
        return;  
        System.out.println("This line should  
cause a mistake");  
    }  
}
```

Preconditions, Postconditions, and Class Invariants

- ▶ Careful programmers use assert statements to state pre- and post-conditions.
 - ▶ That way, if they are wrong, the code stops, rather than doing something stupid or dangerous.

- ▶ Assert $k > 0$

```
sum = 0;
for (i = 0; i < k; i++) {
    sum = sum + i;
}
```

- ▶ Assert $i == k$

Loop Invariant

- ▶ loop invariants: a way to understand what a loop does
- ▶ Loop invariant: property that is true:
 - ▶ First time entering the loop
 - ▶ At the end of each round of loop (iteration)
 - ▶ By implication, at the end of the loop
 - ▶ start (invariant holds);
 - ▶ iterate (invariant holds) =>
 - ▶ invariant holds at exit as well

Loop Invariant Example

```
sum = 0;
for (i = 0; i < k; i++) {
    sum = sum + i;
}
```

- ▶ Loop invariant examples:
 1. i is greater than or equal to 0.
 2. k does not change during the loop
 3. sum contains the sum of values from 0 to $i-1$
- ▶ For code safety, use asserts to capture what you believe to be key invariants in the code.
 - ▶ For example, if you believe that $i == k$ after the loop, write it as an assert.
 - ▶ If you are wrong, the program will fail.

Defensive Programming

- ▶ Defensive programming is a practice where developers anticipate failures in their code
 - ▶ then add supporting code to detect, isolate, and in some cases, recover from the anticipated failure.
- ▶ Defensive programming operates on the premise that bugs exist in our code, so we should identify ways to more easily identify or counteract these problems.
- ▶ Several techniques have been proposed:
 - ▶ Analyze your style and discover where/when you insert bugs in your code
 - ▶ Build your own set of styles

<http://www.drdobbs.com/defensive-programming/184401915>

Defensive Programming

Programming styles - Put the String literal first

```
1  // Bad
2  if (variable.equals("literal")) { ... }
3
4  // Good
5  if ("literal".equals(variable)) { ... }
```

- ▶ It's just never a bad idea to prevent the occasional `NullPointerException` by putting the String literal on the left side of an `equals()` comparison

<https://blog.jooq.org/2015/08/11/top-10-useful-yet-paranoid-java-programming-techniques/>

Defensive Programming

Programming styles - Don't trust that "-1"

- ▶ The Javadoc of [String.indexOf\(\)](#) clearly states that...
 - ▶ the index of the first occurrence of the character in the character sequence represented by this object [is returned], or -1 if the character does not occur.

```
1 // Bad
2 if (string.indexOf(character) != -1) { ... }
3
4 // Good
5 if (string.indexOf(character) >= 0) { ... }
```

- ▶ -1 can be taken for granted
- ▶ Perhaps they'll need *ANOTHER* encoding at some point in time to say, the otherString would have been contained if checked case-insensitively... Perhaps a good case for returning -2? Who knows

Defensive Programming

Programming styles - Check for null AND length

```
1 // Bad
2 if (array.length > 0) { ... }
3
4 // Good
5 if (array != null && array.length > 0) { ... }
```

- ▶ Whenever you have a collection, array, etc., make sure it's present AND not empty.
- ▶ Other possible techniques
 - ▶ Put you methods and variable as final,
 - ▶ Use always bracket in if -- to avoid, e.g.,
if (a>b);
System.out.println("a is greather than b");

Defensive Programming

Programming styles - Assert statements

- ▶ Assert statements are a way to state assumptions about the code.
- ▶ Code will stop execution if assertion is false

```
a = -1; b = 4; c = 4;
assert (a > 0 && b > 0 && c > 0);
if (a + b > c || a + c > b || b + c > a) {
    System.out.println("it is a triangle");
}
else {
    System.out.println("it is not a triangle");
}
```

```
javac SwitchDemo.java
java -ea SwitchDemo
```

To enable the assertion, you have to run the code with `-ea`

```
Exception in thread "main" java.lang.AssertionError
    at SwitchDemo.main(SwitchDemo.java:153)
```

Performance Optimization

▶ General Principles

- ▶ **Optimizing for performance** can often **reduce readability and maintainability**.
- ▶ **Correctness and maintainability** should always come **before performance**.
- ▶ The **only real exception** is in **real-time systems**, where timing constraints are critical.

▶ Common Mistakes

- ▶ **Focusing on performance too early:** premature optimization wastes time.
- ▶ The **first goal** is to write a program that is **correct, clear, and easy to maintain**.
- ▶ In many cases, performance will naturally be sufficient, avoiding unnecessary work.
- ▶ **Optimizing everything** without measurement leads to complexity and errors.
 - ▶ Most parts of a program are executed **only a few times** and do **not need optimization**.

Performance Optimization (2)

▶ Measuring and Refining

- ▶ After making any changes, **measure and profile again** to ensure that performance has improved.
- ▶ **Poor performance** is often a symptom of **bad design or unclear code**. Simplifying and clarifying the design often results in better performance.
- ▶ Whenever possible, **modularize** the program so that optimizations are **isolated** in specific components, while most of the code remains clear and easy to maintain.

▶ Cost-Benefit Analysis

- ▶ Always perform a **cost-benefit evaluation** before starting performance optimization.
- ▶ **Programmer time** is much more expensive than **machine time**. In many cases, it is more economical to **improve the hardware** rather than rewrite complex code.
- ▶ Consider the trade-offs: every optimization may **reduce maintainability** and **increase the risk of new errors**.

▶ Good Practices

- ▶ This warning about performance optimization **does not justify writing inefficient or bloated code**.
- ▶ **Good design choices and sound programming practices** usually lead to software that is **correct, maintainable, and sufficiently fast** without premature optimization.

Refactoring

- ▶ As you develop software, you will **learn about both programming and the problem domain**.
- ▶ **Programming is similar to writing in natural language:** your first draft is rarely perfect.
- ▶ However, in programming, you usually **cannot change the external interface or observable behavior** of a module
 - ▶ Other modules may depend on it.
 - ▶ Therefore, improvements must often be made **internally**, without altering what the software *does*.
- ▶ **Refactoring**
 - ▶ The term **refactoring** was popularized by **Martin Fowler (1999)**.
 - ▶ It means **improving the internal structure of the code without changing its external behavior**.
 - ▶ It focuses on making code **cleaner, clearer, and easier to maintain**.
 - ▶ Refactoring is an essential part of **continuous software improvement**.

Refactoring (2)

- ▶ **Symptoms That Code Can Be Improved**
 - ▶ **Duplicated code:** unnecessary repetition; increases maintenance effort.
 - ▶ **Long method:** overly large methods should be divided into smaller, cohesive ones.
 - ▶ **Large class:** same issue as long methods; often indicates poor modularization.
 - ▶ **Switch statements:** can often be replaced with **polymorphism** for better clarity.
 - ▶ **Feature envy:** a method depends more on another class than on its own; misplaced functionality.
 - ▶ **Inappropriate intimacy:** a class accesses many private details of another class, violating encapsulation.
- ▶ According to Fowler (1999), refactoring is “a change made to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behavior.”
- ▶ Possible refactorings:
 - ▶ **Extract method:** A process that turns a code fragment into its own method and calls the method.
 - ▶ **Substitute algorithm:** It replaces the body of a method with a new algorithm that is clearer, and returns the same result.
 - ▶ **Move method:** A process that moves an algorithm from one class to another where it makes more sense.
 - ▶ **Extract class:** A process that divides a class into two classes.

Building Maintainable Software
*Ten Guidelines for Future-Proof
Code* - The Java edition (ISBN
print: 978-1-4919-5352-5, ISBN
eBook: 978-1-4919-5348-8),
[O'Reilly](#), 2016



Implementation - Building Maintainable Software



Francesco Guerra
francesco.guerra@unimore.it

What Is Maintainability?

- ▶ Maintainability (how easily a system can be modified) is one characteristic of software quality.
- ▶ Four Types of Software Maintenance
 - ▶ *Corrective maintenance*: Bugs are discovered and have to be fixed
 - ▶ *Adaptive maintenance*: The system has to be adapted to changes in the environment in which it works
 - ▶ *Perfective maintenance*: Users of the system (and/or other stakeholders) have new or changed requirements
 - ▶ *Preventive maintenance*: Ways are identified to increase quality or prevent future bugs from occurring

Three Principles of the Guidelines

1. Maintainability benefits most from adhering to simple guidelines.
2. Maintainability is not an afterthought, but should be addressed from the very beginning of a development project. Every individual contribution counts.
3. Some violations are worse than others. The more a software system complies with the guidelines, the more maintainable it is.

Maintainability Guidelines

1. Write short units of code
 - ▶ Shorter units (that is, methods and constructors) are easier to analyze, test, and reuse.
2. Write simple units of code
 - ▶ Units with fewer decision points are easier to analyze and test.
3. Write code once
 - ▶ Duplication of source code should be avoided at all times, since changes will need to be made in each copy. Duplication is also a source of regression bugs.
4. Keep unit interfaces small
 - ▶ Units (methods and constructors) with fewer parameters are easier to test and reuse.
5. Separate concerns in modules
 - ▶ Modules (classes) that are loosely coupled are easier to modify and lead to a more modular system.

Maintainability Guidelines

6. Couple architecture components loosely
 - ▶ Top-level components of a system that are more loosely coupled are easier to modify and lead to a more modular system.
7. Keep architecture components balanced
 - ▶ A well-balanced architecture enables easy modification through separation of concerns.
8. Keep your codebase small
 - ▶ A large system is difficult to maintain, because more code needs to be analyzed, changed, and tested. Also, maintenance productivity per line of code is lower in a large system than in a small system.
9. *Automate development pipeline and tests*
 - ▶ Automated tests enable near-instantaneous feedback on the effectiveness of modifications. Manual tests do not scale.
10. *Write clean code*
 - ▶ Having irrelevant artifacts such as TODOs and dead code in your codebase makes it more difficult for new team members to become productive. Therefore, it makes maintenance less efficient.

1. Write Short Units of Code

- ▶ **Limit the length of code units to 15 lines of code.**
- ▶ **Do this by not writing units that are longer than 15 lines of code in the first place, or by splitting long units into multiple smaller units until each unit has at most 15 lines of code.**
- ▶ **This improves maintainability because small units are easy to understand, easy to test, and easy to reuse.**

Motivation

▶ Short Units Are Easy to Test

- ▶ Code with a **single responsibility** is easier to test.
- ▶ In general, **short units** (functions, methods, modules) perform **only one task**.
- ▶ A unit that handles **one responsibility** allows tests to be
 - ▶ **Isolated**, focusing on one behavior only, and
 - ▶ **Simple**, since there are fewer dependencies.
- ▶ Small, cohesive units lead to **better test coverage** and **faster debugging**.

▶ Short Units Are Easy to Analyze

- ▶ It takes **less time** to read and understand all the code in a short unit.
- ▶ This makes it easier to **analyze** how the unit works internally.
- ▶ The benefit becomes most evident during **maintenance**, when you need to understand existing code before making changes.
- ▶ Maintenance begins **as soon as the project starts**, so writing short, well-defined units is always a good investment.

Motivation (2)

▶ Short Units Are Easy to Reuse

- ▶ A unit should always be **invoked by at least one other method**
 - ▶ Otherwise, it is **dead code** and serves no purpose.
- ▶ **Long units** often contain **specific details** or **multiple combined functions**, which makes them **harder to reuse** in different contexts.
- ▶ **Short units** tend to be **more general and focused**, making them easier to adapt and apply elsewhere.
- ▶ Reusing existing units reduces **duplication** and helps keep the **total code volume low** improving maintainability and consistency across the system.

How to Apply the Guideline

- ▶ We are implementing a java version of the Pacman game

```
public void start() {  
    if (inProgress) {  
        return;  
    }  
    inProgress = true;  
    // Update observers if player died:  
    if (!isAnyPlayerAlive()) {  
        for (LevelObserver o : observers) {  
            o.levelLost();  
        }  
    }  
    // Update observers if all pellets eaten:  
    if (remainingPellets() == 0) {  
        for (LevelObserver o : observers) {  
            o.levelWon();  
        }  
    }  
}
```

16 lines of code

Refactoring technique: Extract Method

- ▶ This technique is applied to extract a method from the former snippet

```
public void start() {  
    if (inProgress) {  
        return;  
    }  
    inProgress = true;  
    updateObservers();  
}  
  
private void updateObservers() {  
    // Update observers if player died:  
    if (!isAnyPlayerAlive()) {  
        for (LevelObserver o : observers) {  
            o.levelLost();  
        }  
    }  
    // Update observers if all pellets eaten:  
    if (remainingPellets() == 0) {  
        for (LevelObserver o : observers) {  
            o.levelWon();  
        }  
    }  
}
```

Refactoring technique: Replace Method with Method Object

- ▶ Sometimes, when refactoring, you may want to **extract a new method** that accesses **local variables** from its original scope.
- ▶ It is always possible to **pass those local variables as parameters** to the extracted method.
- ▶ However, this can result in **long parameter lists**, which are undesirable because they
 - ▶ make the method harder to read,
 - ▶ increase coupling between methods, and
 - ▶ complicate future changes.
- ▶ **Return values** can also create difficulties — for example, in **Java**, a method can **return only a single value**, which may require creating wrapper objects or using data structures to return multiple results.

Example

Extract Method

18 lines of code

```
public Board createBoard(Square[][] grid) {  
    assert grid != null;  
  
    Board board = new Board(grid);  
  
    int width = board.getWidth();  
    int height = board.getHeight();  
    for (int x = 0; x < width; x++) {  
        for (int y = 0; y < height; y++) {  
            Square square = grid[x][y];  
            for (Direction dir : Direction.values()) {  
                int dirX = (width + x + dir.getDeltaX()) % width;  
                int dirY = (height + y + dir.getDeltaY()) % height;  
                Square neighbour = grid[dirX][dirY];  
                square.link(neighbour, dir);  
            }  
        }  
    }  
  
    return board;  
}
```

Lines to extract and refactor

Method with 7 parameters

```
private void setLink(Square square, Direction dir, int x, int y, int width,  
    int height, Square[][] grid) {  
    int dirX = (width + x + dir.getDeltaX()) % width;  
    int dirY = (height + y + dir.getDeltaY()) % height;  
    Square neighbour = grid[dirX][dirY];  
    square.link(neighbour, dir);  
}
```

```
class BoardCreator {
    private Square[][] grid;
    private Board board;
    private int width;
    private int height;
```

we created a new class that will take over the role of createBoard, the method we are refactoring

Example (2)

```
BoardCreator(Square[][] grid) {
    assert grid != null;
    this.grid = grid;
    this.board = new Board(grid);
    this.width = board.getWidth();
    this.height = board.getHeight();
}
```

```
public Board createBoard(Square[][] grid) {
    return new BoardCreator(grid).create();
}
```

Three local variables (board, width, height) and one parameter (grid) of the createBoard method have been turned into (private) fields of the new class.

```
Board create() {
    for (int x = 0; x < width; x++) {
        for (int y = 0; y < height; y++) {
            Square square = grid[x][y];
            for (Direction dir : Direction.values()) {
                setLink(square, dir, x, y);
            }
        }
    }
    return this.board;
}
```

The four lines of the innermost for loop now appear in a new method, setLink, that has four parameters, not seven

```
private void setLink(Square square, Direction dir, int x, int y) {
    int dirX = (width + x + dir.getDeltaX()) % width;
    int dirY = (height + y + dir.getDeltaY()) % height;
    Square neighbour = grid[dirX][dirY];
    square.link(neighbour, dir);
}
}
```

Common Objections to Writing Short Units

“Writing short units means having more units, and therefore more method calls. That will never perform.”

- ▶ Do not sacrifice maintainability for performance.
- ▶ Optimize only when **solid performance tests** show a real issue and prove that optimization **makes a measurable difference**.

“Code becomes harder to read when spread out over multiple units.”

- ▶ Write code that is **easy to read and understand**, not only for others, but also for your **future self**.
- ▶ Clear structure and meaningful naming often make code **more readable**, not less.

“My unit really cannot be split up.”

- ▶ Some constructs, such as a **well-structured switch statement** in Java, may be valid exceptions.
- ▶ When a refactoring seems possible but doesn't make sense, **rethink the overall architecture**: the problem may lie there, not in the unit itself.

2. Write Simple Units of Code

- ▶ **Keep the number of branch points per unit below four.**
 - ▶ Do this by **splitting complex units** into simpler ones and **avoiding deeply nested logic**.
- ▶ This improves **maintainability**: fewer branch points make code **easier to modify and test**.
- ▶ When code becomes too complex, **modifications become risky and time-consuming**, and testing those changes becomes extremely difficult.
- ▶ To keep software maintainable, we have to **set limits on code complexity**.
- ▶ **Measuring Complexity**
 - ▶ The most common metric is **Cyclomatic Complexity (McCabe Complexity)**.
 - ▶ It represents the **number of independent execution paths** in a unit.
 - ▶ The **minimum number of tests** to cover all paths is **equal to the number of branch points plus one**.
 - ▶ **Monitoring and controlling this value ensures that each unit remains testable, predictable, and maintainable.**

Motivation

- ▶ A simple unit is easier to understand, and thus modify, than a complex one.
- ▶ Simple units ease testing.

How to Apply the Guideline

- ▶ In Java the following statements and operators count as branch points:
 - ▶ If / case
 - ▶ ? / && / ||
 - ▶ While / For
 - ▶ Catch
- ▶ A **complex unit** may consist of multiple **code blocks glued together**, where total complexity is the **sum of each block's complexity**.
 - ▶ **Long chains of if-then-else** or large switch statements.
 - ▶ **Deeply nested conditionals**, which reduce clarity and increase testing effort.
- ▶ **Refactoring Strategies**
 - ▶ **Extract Method**: split complex logic into smaller, independent methods.
 - ▶ **Replace Conditional with Polymorphism**: use object-oriented design instead of long conditional chains.
 - ▶ **Replace Nested Conditional with Guard Clauses**: simplify structure by handling special cases early and clearly.

Example Replace Conditional with Polymorphism pattern

```
public List<Color> getFlagColors(Nationality nationality) {  
    List<Color> result;  
    switch (nationality) {  
        case DUTCH:  
            result = Arrays.asList(Color.RED, Color.WHITE, Color.BLUE);  
            break;  
        case GERMAN:  
            result = Arrays.asList(Color.BLACK, Color.RED, Color.YELLOW);  
            break;  
        case BELGIAN:  
            result = Arrays.asList(Color.BLACK, Color.YELLOW, Color.RED);  
            break;  
        case FRENCH:  
            result = Arrays.asList(Color.BLUE, Color.WHITE, Color.RED);  
            break;  
        case ITALIAN:  
            result = Arrays.asList(Color.GREEN, Color.WHITE, Color.RED);  
            break;  
        case UNCLASSIFIED:  
        default:  
            result = Arrays.asList(Color.GRAY);  
            break;  
    }  
    return result;  
}
```

Example *Replace Conditional with Polymorphism* pattern (2)

```
public interface Flag {  
    List<Color> getColors();  
}
```

```
public class DutchFlag implements Flag {  
    public List<Color> getColors() {  
        return Arrays.asList(Color.RED, Color.WHITE, Color.BLUE);  
    }  
}
```

```
public class ItalianFlag implements Flag {  
    public List<Color> getColors() {  
        return Arrays.asList(Color.GREEN, Color.WHITE, Color.RED);  
    }  
}
```

```
private static final Map<Nationality, Flag> FLAGS =  
    new HashMap<Nationality, Flag>();
```

```
static {  
    FLAGS.put(DUTCH, new DutchFlag());  
    FLAGS.put(GERMAN, new GermanFlag());  
    FLAGS.put(BELGIAN, new BelgianFlag());  
    FLAGS.put(FRENCH, new FrenchFlag());  
    FLAGS.put(ITALIAN, new ItalianFlag());  
}
```

```
public List<Color> getFlagColors(Nationality nationality) {  
    Flag flag = FLAGS.get(nationality);  
    flag = flag != null ? flag : new DefaultFlag();  
    return flag.getColors();  
}
```

Replace Nested Conditional with Guard Clauses

- ▶ A method has conditional behavior that does not make clear what the normal path of execution is

```
double getPayAmount() {
    double result;
    if (_isDead) result = deadAmount();
    else {
        if (_isSeparated) result = separatedAmount();
        else {
            if (_isRetired) result = retiredAmount();
            else result = normalPayAmount();
        };
    }
    return result;
};
```

Isolate all special checks and edge cases into separate clauses and place them before the main checks. Ideally, you should have a "flat" list of conditionals, one after the other.

Why This Is Better

- Reduces **nesting depth** and improves readability.
- Makes the **normal flow** of logic clearer.
- Each special case is handled immediately.
- Easier to **test and maintain**.

```
double getPayAmount() {
    if (_isDead) return deadAmount();
    if (_isSeparated) return separatedAmount();
    if (_isRetired) return retiredAmount();
    return normalPayAmount();
};
```

Example Replace Nested Conditional with Guard Clauses

```
public static int calculateDepth(BinaryTreeNode<Integer> t, int n) {  
    int depth = 0;  
    if (t.getValue() == n) {  
        return depth;  
    } else {  
        if (n < t.getValue()) {  
            BinaryTreeNode<Integer> left = t.getLeft();  
            if (left == null) {  
                throw new TreeException("Value not found in tree!");  
            } else {  
                return 1 + calculateDepth(left, n);  
            }  
        } else {  
            BinaryTreeNode<Integer> right = t.getRight();  
            if (right == null) {  
                throw new TreeException("Value not found in tree!");  
            } else {  
                return 1 + calculateDepth(right, n);  
            }  
        }  
    }  
}
```

- ▶ Given a binary search tree root node and an integer, the calculateDepth method determines whether the integer occurs in the tree.

Example *Replace Nested Conditional with Guard Clauses* (2)

```
public static int calculateDepth(BinaryTreeNode<Integer> t, int n) {  
    int depth = 0;  
    if (t.getValue() == n)  
        return depth;  
    if (n < t.getValue() && t.getLeft() != null)  
        return 1 + calculateDepth(t.getLeft(), n);  
    if (n > t.getValue() && t.getRight() != null)  
        return 1 + calculateDepth(t.getRight(), n);  
    throw new TreeException("Value not found in tree!");  
}
```

- ▶ With the recursive implementation the code is more compact, but the complexity is the same

Example *Replace Nested Conditional with Guard Clauses* (3)

```
public static int calculateDepth(BinaryTreeNode<Integer> t, int n) {  
    int depth = 0;  
    if (t.getValue() == n)  
        return depth;  
    else  
        return traverseByValue(t, n);  
}
```

```
private static int traverseByValue(BinaryTreeNode<Integer> t, int n) {  
    BinaryTreeNode<Integer> childNode = getChildNode(t, n);  
    if (childNode == null) {  
        throw new TreeException("Value not found in tree!");  
    } else {  
        return 1 + calculateDepth(childNode, n);  
    }  
}
```

```
private static BinaryTreeNode<Integer> getChildNode(  
    BinaryTreeNode<Integer> t, int n) {  
    if (n < t.getValue()) {  
        return t.getLeft();  
    } else {  
        return t.getRight();  
    }  
}
```

- ▶ the methods are easier to understand
- ▶ they are easier to test in isolation.

Common Objections to Writing Simple Units of Code

“Our domain is very complex, and therefore high code complexity is unavoidable.”

- ▶ The **more complex** the domain, the **more effort** developers must invest to create **technically simple solutions**.
- ▶ Simplicity requires design skill and discipline: but **it is possible**.
- ▶ Strive to make **code structure simpler than the problem itself**.

“Replacing one method with McCabe 15 by three methods with McCabe 5 each doesn’t help: overall complexity is still 15.”

- ▶ Mathematically, the **total control-flow complexity** remains the same.
- ▶ But from a **maintainability perspective**, this refactoring has clear advantages:
 - ▶ Each method is **easier to test** individually.
 - ▶ Each method is **easier to read and understand**.
 - ▶ The system becomes **more modular and predictable**.

3. Write Code Once

- ▶ **Avoid copying and pasting code** between different parts of the program.
- ▶ Instead, aim to **write reusable and generic code**, or **call existing methods** whenever possible.
- ▶ This practice improves **maintainability** and **consistency**.
- ▶ When code is copied, any **bug or required change** must be fixed in **multiple places**, which is **inefficient, error-prone**, and often leads to inconsistencies.
- ▶ Refactoring to **centralize common logic** reduces duplication and simplifies maintenance.

Example

```
public class CheckingAccount {
    private int transferLimit = 100;

    public Transfer makeTransfer(String counterAccount, Money amount)
        throws BusinessException {
        // 1. Check withdrawal limit:
        if (amount.greaterThan(this.transferLimit)) {
            throw new BusinessException("Limit exceeded!");
        }
        // 2. Assuming result is 9-digit bank account number, validate 11-test:
        int sum = 0;
        for (int i = 0; i < counterAccount.length(); i++) {
            sum = sum + (9-i) * Character.getNumericValue(
                counterAccount.charAt(i));
        }
        if (sum % 11 == 0) {
            // 3. Look up counter account and make transfer object:
            CheckingAccount acct = Accounts.findAcctByNumber(counterAccount);
            Transfer result = new Transfer(this, acct, amount);
            return result;
        } else {
            throw new BusinessException("Invalid account number!");
        }
    }
}
```

makeTransfer checks whether the number of the account to transfer the money to complies with a checksum

Example (2)

```

public class SavingsAccount {
    CheckingAccount registeredCounterAccount;

    public Transfer makeTransfer(String counterAccount, Money amount)
        throws BusinessException {
        // 1. Assuming result is 9-digit bank account number, validate 11-test:
        int sum = 0; ❶
        for (int i = 0; i < counterAccount.length(); i++) {
            sum = sum + (9 - i) * Character.getNumericValue(
                counterAccount.charAt(i));
        }
        if (sum % 11 == 0) {
            // 2. Look up counter account and make transfer object:

            CheckingAccount acct = Accounts.findAcctByNumber(counterAccount);
            Transfer result = new Transfer(this, acct, amount); ❷
            // 3. Check whether withdrawal is to registered counter account:
            if (result.getCounterAccount().equals(this.registeredCounterAccount))
            {
                return result;
            } else {
                throw new BusinessException("Counter-account not registered!");
            }
        } else {
            throw new BusinessException("Invalid account number!!");
        }
    }
}

```

► Now assume the bank introduces a new account type, called a savings account.

- A savings account does not have a transfer limit,
- but money can only be transferred to one particular (fixed) checking account.

Example(3)

- ▶ If a bug is discovered in the implementation of the 11-test
 - ▶ This bug now needs to be fixed in both duplicates.
- ▶ Resist the temptation of gaining a short-term advantage by copying and pasting code. For every future adjustment to either duplicate, you will need to revisit all duplicates.

Motivation

▶ Duplicated Code Is Harder to Analyze

- ▶ The main problem with duplication is **uncertainty**: you don't know whether **other copies** of the same code exist, **how many** there are, or **where** they are located.
- ▶ This makes it difficult to **understand the full behavior** of the program.
- ▶ Analyzing duplicated code becomes time-consuming and error-prone, as each copy might evolve differently over time.

▶ Duplicated Code Is Harder to Modify

- ▶ When code is duplicated, **bug fixes and updates** must be applied in **every copy**.
- ▶ Developers must first **realize that a duplicate exists** before they can modify it.
- ▶ Overlooking one duplicate can lead to **regression bugs**: functionality that previously worked suddenly fails because an identical but unfixed copy remains in use.
- ▶ Eliminating duplication ensures **consistent behavior** and **reduces maintenance effort**.

How to Apply the Guideline

- ▶ The Extract Method refactoring technique is the workhorse that solves many duplication problems

```
public class CheckingAccount {  
    private int transferLimit = 100;  
  
    public Transfer makeTransfer(String counterAccount, Money amount)  
        throws BusinessException {  
        // 1. Check withdrawal limit:  
        if (amount.greaterThan(this.transferLimit)) {  
            throw new BusinessException("Limit exceeded!");  
        }  
        if (Accounts.isValid(counterAccount)) { ❶  
            // 2. Look up counter account and make transfer object:  
            CheckingAccount acct = Accounts.findAcctByNumber(counterAccount);  
            Transfer result = new Transfer(this, acct, amount); ❷  
            return result;  
        } else {  
            throw new BusinessException("Invalid account number!");  
        }  
    }  
}
```

The extracted fragment has to be put in a third class.

This class runs the risk of becoming a hodgepodge of unrelated methods.

This leads to a *large class smell* and *tight coupling*.

The Extract Superclass Refactoring Technique

- ▶ The *Extract Superclass* refactoring technique uses this feature by extracting a fragment of code lines not just to a method, but to a new class that is the superclass of the original class.

```
public class Account {
    public Transfer makeTransfer(String counterAccount, Money amount)
        throws BusinessException {
        // 1. Assuming result is 9-digit bank account number, validate 11-test:
        int sum = 0; ❶
        for (int i = 0; i < counterAccount.length(); i++) {
            sum = sum + (9 - i) * Character.
                getNumericValue(counterAccount.charAt(i));
        }
        if (sum % 11 == 0) {
            // 2. Look up counter account and make transfer object:
            CheckingAccount acct = Accounts.findAcctByNumber(counterAccount);
            Transfer result = new Transfer(this, acct, amount); ❷
            return result;
        } else {
            throw new BusinessException("Invalid account number!");
        }
    }
}
```

Note that this method has two responsibilities: we could divide it into two methods

Example: Extract Superclass Refactoring Technique

```
public class CheckingAccount extends Account {
    private int transferLimit = 100;

    @Override
    public Transfer makeTransfer(String counterAccount, Money amount)
        throws BusinessException {
        if (amount.greaterThan(this.transferLimit)) {
            throw new BusinessException("Limit exceeded!");
        }
        return super.makeTransfer(counterAccount, amount);
    }
}

public class SavingsAccount extends Account {
    CheckingAccount registeredCounterAccount;

    @Override
    public Transfer makeTransfer(String counterAccount, Money amount)
        throws BusinessException {
        Transfer result = super.makeTransfer(counterAccount, amount);
        if (result.getCounterAccount().equals(this.registeredCounterAccount)) {
            return result;
        } else {
            throw new BusinessException("Counter-account not registered!");
        }
    }
}
```

Common Objections to Avoiding Code Duplication

“Copying and pasting code from another codebase is not a problem because it will not create a duplicate in the current system.”

- ▶ That may be true for the **current codebase**, but the **original codebase continues to evolve and be maintained**.
- ▶ Future **bug fixes or improvements** in the source project will not automatically propagate, leaving your copied code **out of sync and harder to maintain**.

“Duplication is unavoidable because we need slight variations of common functionality.”

- ▶ Identify and extract the **common parts** shared by all variants.
- ▶ Move shared behavior into a **common superclass or utility component**.
- ▶ Model the differences so they are **explicit, isolated, and testable** — for example, through **inheritance, composition, or configuration parameters**.
- ▶ This keeps the code **flexible, maintainable, and free of hidden duplication**.

Common Objections to Avoiding Code Duplication(2)

“This code will never, ever change, so there is no harm in duplicating it.”

- ▶ **Nothing in software stays unchanged.**
- ▶ **Functional requirements may evolve because of new users or new behaviors.**
- ▶ **The organization may change:** ownership, responsibilities, or development approach can shift.
- ▶ **The technology stack may change:** operating systems, libraries, or frameworks are regularly updated.
- ▶ **The code itself may change:** through bug fixes, refactoring, or even stylistic improvements.
- ▶ **Duplicated code makes it harder to apply these inevitable changes consistently.**

“Unit tests will detect any problems caused by duplicated code.”

- ▶ **Relying on tests to catch problems after the fact is a weak strategy.**
- ▶ **It is better to avoid the problem altogether by eliminating duplication now.**
- ▶ **Prevention is always cheaper and safer than detection and correction later in development.**

4. Keep Unit Interfaces Small

- ▶ **Keep the number of parameters per unit to four or fewer.**
- ▶ When a method requires many parameters, it usually indicates that the unit **does too much** or that **data is poorly structured**.
- ▶ To simplify, **extract related parameters into an object** (for example, a CustomerInfo or Config object).
- ▶ This approach improves **readability, reusability, and maintainability**.
- ▶ Fewer parameters make a method **easier to understand, less error-prone, and simpler to extend later**.

Example

```
/**
 * Renders a single square on the given graphics context on the specified
 * rectangle.
 *
 * @param square
 *           The square to render.
 * @param g
 *           The graphics context to draw on.
 * @param x
 *           The x position to start drawing.
 * @param y
 *           The y position to start drawing.
 * @param w
 *           The width of this square (in pixels).
 * @param h
 *           The height of this square (in pixels).
 */
private void render(Square square, Graphics g, int x, int y, int w, int h) {
    square.getSprite().draw(g, x, y, w, h);
    for (Unit unit : square.getOccupants()) {
        unit.getSprite().draw(g, x, y, w, h);
    }
}
```

- ▶ x,y,w, h variables are related, and the render method does not manipulate these variables independently,
 - ▶ it makes sense to group them into an object of type Rectangle.

Motivation

- ▶ Methods with small interfaces keep their context simple and thus are easier to understand.
- ▶ They are easier to reuse and modify because they do not depend on too much external input.

How to Apply the Guideline

- ▶ Large interfaces are usually **not the real problem** — they are a **code smell** that indicates a **deeper maintainability issue**.
- ▶ **Solution:** group related parameters into a **single object**.
 - ▶ Such objects are often called **Data Transfer Objects (DTOs)** or **Parameter Objects**.
- ▶ This makes interfaces **simpler, more readable, and easier to extend**.

```
public void buildAndSendMail(MailMan m, String firstName, String lastName,
    String division, String subject, MailFont font, String message1,
    String message2, String message3) {
    // Format the email address
    String mId = firstName.charAt(0) + "." + lastName.substring(0, 7) + "@"
        + division.substring(0, 5) + ".compa.ny";
    // Format the message given the content type and raw message
    MailMessage mMessage = formatMessage(font,
        message1 + message2 + message3);
    // Send message
    m.send(mId, subject, mMessage);
}
```

The method clearly has too many responsibilities

How to Apply the Guideline (2)

```
public void buildAndSendMail(MailMan m, MailAddress mAddress,
    MailBody mBody) {
    // Build the mail
    Mail mail = new Mail(mAddress, mBody);
    // Send the mail
    m.sendMail(mail);
}

private class Mail {
    private MailAddress address;
    private MailBody body;

    private Mail(MailAddress mAddress, MailBody mBody) {
        this.address = mAddress;
        this.body = mBody;
    }
}

private class MailBody {
    String subject;
    MailMessage message;

    public MailBody(String subject, MailMessage message) {
        this.subject = subject;
        this.message = message;
    }
}
```

Refactoring Pattern: *Introduce Parameter Object*

Goal: Replace multiple related parameters with a single object.

Benefits:

- Reduces method signatures' length.
- Groups related data logically.
- Improves maintainability and reusability.

And so on for the MailAddress class

Common Objections to Keeping Unit Interfaces Small

“The parameter object I introduced now has a constructor with too many parameters.”

- ▶ Do not abandon the *Introduce Parameter Object* refactoring.
- ▶ Rethink the **structure** of the object and how it relates to other parts of the code.
- ▶ **Group related parameters** into smaller **value objects** (e.g., Point(x, y) instead of separate coordinates).
- ▶ If there are many optional arguments, apply **Replace Constructor with Builder**.

“The framework interface has nine parameters. How can I implement it without violating unit interface limits?”

- ▶ Some **interface violations** are **unavoidable**, especially with external frameworks.
- ▶ **Isolate** these violations using **wrappers** or **adapters**.
- ▶ Provide a **simpler internal interface** (Facade or Parameter Object) for the rest of the system.
- ▶ This limits the impact and keeps your own code **clean and maintainable**.

5. Separate Concerns in Modules

- ▶ **Avoid large modules** to achieve **loose coupling** between components.
- ▶ Do this by **assigning clear responsibilities** to separate modules and **hiding implementation details** behind well-defined interfaces.
- ▶ This approach improves **maintainability**:
 - ▶ In a **loosely coupled** codebase, changes are **easier to understand, isolate, and execute**.
 - ▶ In a **tightly coupled** codebase, a small change can have **unpredictable side effects**.
- ▶ In object-oriented languages such as **Java**, the concept of a **module** often corresponds to a **class** or a **package**.

Motivation

- ▶ Small, Loosely Coupled Modules Allow Developers to Work on Isolated Parts of the Codebase
- ▶ Small, Loosely Coupled Modules Ease Navigation Through the Codebase

How to Apply the Guideline

- ▶ Split Classes to Separate Concerns
- ▶ Hide Specialized Implementations Behind Interfaces
- ▶ Replace Custom Code with Third-Party Libraries/Frameworks

Common Objections to Separating Concerns

“It doesn’t make sense to use Java interfaces to prevent tight coupling.”

- ▶ Using **interfaces** is indeed a powerful way to improve encapsulation and hide implementations.
- ▶ However, it does **not make sense to define an interface for every class**.
- ▶ As a **rule of thumb**, an interface should be implemented by at least two classes in your codebase.
- ▶ Otherwise, it adds unnecessary complexity without improving flexibility.

“Utility code will always be called from many locations in the codebase.”

- ▶ That is true: **highly generic, reusable code** is often needed.
- ▶ But such code should remain **small, cohesive, and stable**.
- ▶ Before writing your own utilities, check whether a **standard library or framework** already provides the same functionality.
- ▶ Reusing existing, tested components **reduces maintenance and improves reliability**.

6. Couple Architecture Components Loosely

- ▶ Strive to **minimize coupling** between **top-level components** of the system.
- ▶ Do this by **reducing the amount of code** within each module that is **exposed** to – or can be **called by** – modules in other components.
- ▶ Keep **interfaces narrow and well-defined**, exposing only what is necessary.
- ▶ This improves **maintainability** because
 - ▶ components remain **independent**, and
 - ▶ maintenance or modification can be done **in isolation** without side effects.

Motivation

- ▶ Low Component Dependence Allows for Isolated Maintenance
- ▶ Low Component Dependence Separates Maintenance Responsibilities
- ▶ Low Component Dependence Eases Testing

How to Apply the Guideline

- ▶ **Keep component interfaces small and focused:** define them at a high level of abstraction.
- ▶ This limits the **types of requests** that cross component boundaries and prevents dependencies that “**know too much**” about internal implementations.
- ▶ **Avoid throughput code, i.e.,** interface modules that merely **pass calls** to other components.
 - ▶ Throughput code makes testing difficult and reduces design clarity.
 - ▶ If throughput code exists, **analyze and refactor** the affected modules to **eliminate unnecessary delegation** and **restore clear component boundaries**.

Common Objections to Loose Component Coupling

“In the maintenance team, we understand the importance of achieving low component dependence, but we are not granted time to fix it.”

- ▶ Development deadlines are real, and **time for refactoring may be limited**.
- ▶ What matters is the **trade-off**: fix only the dependencies that cause **real problems**.
- ▶ Dependencies should be resolved when they **inhibit testing, analysis, or system stability**.
- ▶ Focus refactoring effort where it **provides measurable maintainability benefits**.

“We have a requirement for a software architecture that puts through calls.”

- ▶ Some architectures **intentionally include an intermediate layer**, such as a **service layer** that collects requests from one side (e.g., the UI) and **forwards them** to another layer.
- ▶ The presence of such a layer is **not a problem** if it promotes **loose coupling**.
- ▶ It should maintain a **clear separation** between incoming and outgoing requests:
 - ▶ The receiving module **must not process the request** itself.
 - ▶ It **must not know** how or where the request is processed (implementation details).
- ▶ When both are true, the service layer acts as a **communication boundary**, not as a **throughput module**.

Example

Bad Example: Throughput Layer (No Separation)

```
class OrderFacade {  
    private OrderService service = new  
    OrderService();  
  
    public void processOrder(Order order) {  
        // Just passes the call  
        service.processOrder(order);  
    }  
}
```

Good Example: Service Layer (Loose Coupling)

```
class OrderController {  
    private final OrderService service;  
  
    public OrderController(OrderService  
    service) {  
        this.service = service;  
    }  
  
    public void  
    handleOrderRequest(OrderRequest req) {  
        // Translates incoming UI request →  
        domain call  
        Order order = convert(req);  
        service.process(order);  
    }  
}
```

7. Keep Architecture Components Balanced

- ▶ **Balance** both the **number** and **relative size** of top-level components in your codebase.
- ▶ Organize the source code so that the number of components is small typically between **six and twelve**.
- ▶ Ensure components are of **comparable size and scope**, each with a **clear and distinct responsibility**.
- ▶ This structure improves **maintainability** because:
 - ▶ Components are **easier to locate and navigate**.
 - ▶ Maintenance tasks can be **performed in isolation** without side effects.
 - ▶ The overall architecture remains **coherent and predictable**.

Motivation

▶ A Good Component Balance

▶ Eases Finding and Analyzing Code

- ▶ A clear **organization** into components makes it easier to **locate and understand** the code you need to modify.
- ▶ When the number of components is **manageable**, developers can **drill down logically** during analysis.
- ▶ Balanced components provide a **structured path** for navigating the system.

▶ Better Isolates Maintenance Effects

- ▶ A well-balanced architecture ensures **clear functional boundaries** and **proper separation of concerns**.
- ▶ This allows for **isolated maintenance**, where changes in one component have **minimal side effects** on others.
- ▶ Such **isolation** prevents **unexpected regressions** and improves **testability**.
- ▶ Modular, well-defined components are easier to **replace, remove, or test** than those with **mixed responsibilities**.

▶ Separates Maintenance Responsibilities

- ▶ Clear component boundaries also make it easier to **assign maintenance tasks** across different teams.
- ▶ Too **few components** make navigation and ownership unclear.
- ▶ Too **many components** make the system **hard to oversee** as a whole.
- ▶ A balanced number ensures that **responsibilities and dependencies remain clear and manageable**.

How to Apply the Guideline

- ▶ The two principles of component balance are:
 - ▶ The number of top-level system components should ideally be 9, and generally between 6 and 12.
 - ▶ Note that component balance is an *indicator* for a clear component separation, not a goal in itself.
 - ▶ It should follow from the system design and development process.
 - ▶ The division of the system into components should be natural, not forced to nine components for the sake of having nine components.
 - ▶ The components' volume in terms of source code should be roughly equal.

How to Apply the Guideline (2)

▶ Decide on the Right Conceptual Level for Component Grouping

▶ Functional Domain-Based Organization

- ▶ Software systems can be structured around **high-level functional domains**: components that represent **what the system does** for the user.
- ▶ Example components: **Data Retrieval, Invoice Administration, Reporting, Administration**, etc.
- ▶ Each component provides **end-to-end functionality**, from database to frontend.
- ▶ **Advantages**: Developers can analyze code while thinking in **terms of user functionalities**.
- ▶ **Disadvantages**: Developers may need proficiency in **multiple technical areas** (e.g., UI, database, logic).

▶ Technical Specialization-Based Organization

- ▶ Systems can be divided along **technical boundaries**: components that represent **how the system is built**.
- ▶ Example components: **Frontend, Backend, Interfaces, Logging**, etc.
- ▶ **Advantages**: Easier **division of responsibilities** based on technical expertise.
- ▶ **Disadvantages**: Harder to trace **end-to-end functionality** across technical layers.

▶ Clarify and Apply Consistently

- ▶ Once you choose a **component division strategy** apply it **consistently across the entire system**.
- ▶ Consistency helps maintain a **clear architecture** and ensures that the system remains **understandable and maintainable** over time.

Common Objections to Balancing Components

“Our system may seem to have bad component balance, but we’re not having any problems with it.”

- ▶ Component balance is a **spectrum**.
 - ▶ Some **deviation from the ideal** (around nine components of similar size) is acceptable.
- ▶ The goal is not perfection but **clarity and maintainability**.
- ▶ Systems often degrade gradually, so keeping balance under review **prevents future problems**.

“We cannot get component balance right because of entanglement among components.”

- ▶ This points to a **different underlying issue**: technical **dependence** between components.
 - ▶ Such **entanglement** reveals an **improper separation of concerns**.
- ▶ It is more important to **reduce component dependencies** — for example, by
 - ▶ **hiding implementation details behind interfaces**, or
 - ▶ **introducing abstraction layers** to decouple functionality.
- ▶ Once dependencies are reduced, achieving a better **component balance** becomes possible.

8. Keep Your Codebase Small

- ▶ **Minimize the size of your codebase** to reduce complexity and ease maintenance.
- ▶ Do this by **avoiding unnecessary growth** and by **actively reducing** unused or redundant code.
- ▶ Smaller products, projects, and teams are **easier to manage and maintain**, making system evolution **more predictable and less risky**.
- ▶ **What Is a Codebase?**
 - ▶ A **codebase** is a **collection of source code** that:
 - ▶ Is stored in a **single repository**,
 - ▶ Can be **compiled and deployed independently**, and
 - ▶ Is **maintained by one team**.
 - ▶ A **system** has at least one codebase larger systems may have **multiple codebases**:
 - ▶ A **core product** codebase, and
 - ▶ Separate **plugin or customer-specific** codebases maintained independently.

Motivation

- ▶ A Project That Sets Out to Build a Large Codebase Is More Likely to Fail
 - ▶ There is a strong correlation between project size and project risks.
- ▶ Large Systems Have Higher Defect Density
- ▶ Large Codebases Are Harder to Maintain

How to Apply the Guideline

- ▶ All other things being equal, a system that has less *functionality* will be smaller than a system that has more functionality.
- ▶ The *implementation* of that functionality may be either concise or verbose.
- ▶ Therefore, achieving a small codebase first requires keeping the functionality of a system limited, and then requires attention to keep the amount of code limited.
- ▶ Functional Measures
 - ▶ *Fight scope creep*
 - ▶ *Standardize functionality*: consistency in the behavior and interactions of the program.
 - ▶ Avoid the implementation of the same core functionality in multiple, slightly different ways.

How to Apply the Guideline (2)

▶ Do Not Copy and Paste Code

- ▶ Copied code must be **maintained in multiple places**, leading to errors and inefficiency.
- ▶ Promoting **reuse** improves consistency and reduces maintenance effort.

▶ Refactor Existing Code

- ▶ Refactoring has long-term benefits for **maintainability**, but it can also **immediately reduce the size** of the codebase.
- ▶ Refactoring typically involves: **Simplifying** structure, **Removing redundancies** or obsolete code, **Improving reuse** and cohesion.

▶ Use Third-Party Libraries and Frameworks

- ▶ When functionality is already available in a **reliable external library**, use it. Avoid reinventing solutions that are **well-tested and maintained** by others.

▶ Split Up a Large System

- ▶ If the system becomes too large, **divide it into smaller, independent systems**. Each smaller system is **easier to maintain, test, and evolve**.
- ▶ This approach minimizes the complexity and risks associated with **large monolithic systems**.

Common Objections to Keeping the Codebase Small

“I cannot possibly reduce the size of my system, since my programming productivity is measured in added code volume.”

- ▶ Measuring productivity by **lines of code added** is a **bad practice**. It creates a **negative incentive**, encouraging code duplication and poor design.
- ▶ Productivity should be measured by **value delivered**, not by the **amount of code written**.

“I work with a language that is more verbose than others, so I cannot achieve a small codebase.”

- ▶ The **choice of programming language** is often fixed and not under developer control.
- ▶ Even in verbose languages, you can still **reduce code size** by:
 - ▶ **Refactoring** to eliminate redundancy,
 - ▶ **Reusing existing components**.

9. Automate Tests

- ▶ **Write automated tests** for your code using a reliable **test framework**.
- ▶ Automated testing makes development **predictable** and **less risky** by detecting issues early.
- ▶ It ensures that code changes can be **validated continuously** without manual effort.
- ▶ Automated tests increase **confidence** in refactoring and speed up **maintenance activities**.
- ▶ A well-tested codebase is **more stable**, **easier to evolve**, and **safer to extend**.

Motivation

- ▶ **Automated Testing Makes Testing Repeatable**
 - ▶ When a test produces different results at different times, the **fault lies in the code**, not in the test itself.
 - ▶ Automated tests ensure **repeatability** and **reliability** in the testing process.
- ▶ **Automated Testing Makes Development Efficient**
 - ▶ Automated tests can be executed with **minimal effort** compared to manual tests.
 - ▶ This makes development more **productive** and **less error-prone**.
- ▶ **Automated Testing Makes Code Predictable**
 - ▶ A major benefit is the **early detection of regressions** identifying when existing functionality breaks.
- ▶ **Tests Document the Code That Is Tested**
 - ▶ Test scripts contain **assertions** about the expected system behavior. As a result, they serve as **executable documentation** that describes how the code should work.
- ▶ **Writing Tests Makes You Write Better Code**
 - ▶ Writing tests encourages **testable design**: shorter, simpler, and more cohesive units.
 - ▶ Code that is easy to test also tends to be **more modular, loosely coupled, and maintainable**.

How to Apply the Guideline

- ▶ Black Box test
- ▶ White Box test
- ▶ See lecture on this topic

Common Objections to Automating Tests

“Why should we invest in automated tests at all if we still need manual testing?”

- ▶ Automated testing and manual testing **complement each other**.
- ▶ **Automation frees up time** for manual testing by covering routine and repeatable checks.
- ▶ This allows testers to **focus on complex, exploratory, or usability testing**, i.e., areas that cannot be automated effectively.
- ▶ The result is **broader test coverage** and **more efficient use of human effort**.

“I am not allowed to write unit tests because they lower productivity, according to my manager.”

- ▶ In reality, writing unit tests **increases long-term productivity**.
- ▶ Unit tests help identify errors **early**, before they propagate.
- ▶ They **improve system code quality** by shifting focus from “*what the code should do*” to “*what could go wrong*.”
- ▶ Considering how code may fail makes it **more robust and resilient** to unexpected situations.
- ▶ The initial time investment in testing is quickly recovered through **fewer bugs** and **faster maintenance**.

10. Write Clean Code

- ▶ **Write clean, readable, and well-structured code** throughout development.
- ▶ Do this by **not leaving code smells**, such as duplication, long methods, or unclear names, behind after completing your work.
- ▶ Clean code is **easier to understand, test, and modify**, reducing long-term maintenance costs.
- ▶ In short: **clean code is maintainable code**.

Leave No Trace

- ▶ **Leave no unit-level code smells:** refactor long, complex, or unclear methods.
- ▶ **Leave no bad comments:** keep only meaningful, up-to-date explanations.
- ▶ **Leave no code in comments:** remove commented-out code; use version control instead.
- ▶ **Leave no dead code:** delete unused classes, methods, or variables.
- ▶ **Leave no long identifier names:** choose clear but concise names for readability.
- ▶ **Leave no magic constants:** replace unexplained numbers or strings with named constants.
- ▶ **Leave no badly handled exceptions:** manage errors explicitly and meaningfully.

1. .

Common Objections to Writing Clean Code

“We use comments to document the workings of the code.”

- ▶ Comments that **accurately describe** the code can be helpful, especially for **less experienced developers**.
- ▶ In practice, however, **most comments become outdated** over time – not intentionally, but because they are **rarely updated** when the code changes.
- ▶ Outdated comments **mislead developers** and reduce code reliability.
- ▶ Maintaining comment accuracy requires **continuous precision**, which is often neglected.
- ▶ By keeping units **small and simple**, and using **clear, descriptive names**, comments for documentation become **largely unnecessary**.



References

- ▶ Frank Tsui, Orlando Karam, Barbara Bernal, Software engineering (Fifth Edition), Jones & Bartlett Learning
- ▶ Building Maintainable Software - Ten Guidelines for Future-Proof Code - The Java edition (ISBN print: 978-1-4919-5352-5, ISBN eBook: 978-1-4919-5348-8), O'Reilly, 2016

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.